



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
BACHARELADO INTERDISCIPLINAR EM TECNOLOGIA DA INFORMAÇÃO
LABORATÓRIO DE ALGORITMOS E ESTRUTURAS DE DADOS II
KENNEDY REURISON LOPES

FABÍOLA FERREIRA DA SILVA
MARIA HELOÍZE DE QUEIROZ CARVALHO
RUTH CARVALHO SILVA

**RELATÓRIO SOBRE SISTEMA DE CONSULTA EFICIENTE COM TABELA HASH
E FILTRO DE BLOOM**

PAU DOS FERROS - RN

2026

FABÍOLA FERREIRA DA SILVA
MARIA HELOÍZE DE QUEIROZ CARVALHO
RUTH CARVALHO SILVA

**RELATÓRIO SOBRE SISTEMA DE CONSULTA EFICIENTE COM TABELA HASH
E FILTRO DE BLOOM**

Relatório técnico apresentado como requisito parcial para aprovação na disciplina de Laboratório de Algoritmos e Estrutura de Dados II, do curso de Bacharelado em Interdisciplinar em Tecnologia da Informação da Universidade Federal Rural do Semi-Árido - *Campus* Pau dos Ferros.

Professor: Kennedy Reurison Lopes

SUMÁRIO

1. INTRODUÇÃO.....	4
2. OBJETIVOS.....	5
2.2. Objetivo Geral.....	5
2.1. Objetivos Específicos.....	5
3. MATERIAL E MÉTODOS.....	6
4. RESULTADOS E DISCUSSÃO.....	7
5. CONSIDERAÇÕES FINAIS.....	9
6. REFERÊNCIAS.....	10

RESUMO

O presente relatório descreve o desenvolvimento e a análise de desempenho de um Sistema de Verificação de Cadastro de Usuários, projetado para armazenar e consultar grandes volumes de dados de forma eficiente. O objetivo central do trabalho é avaliar o impacto prático e computacional do uso combinado de duas estruturas de dados: uma Tabela Hash, responsável pelo armazenamento e recuperação exata dos elementos, e um Filtro de Bloom, empregado como uma estrutura probabilística auxiliar para acelerar as consultas de existência e reduzir acessos desnecessários à estrutura principal. A metodologia adotada consistiu na implementação manual de ambas as estruturas em linguagem C, sem o uso de bibliotecas prontas. O fluxo de busca foi desenhado para consultar primeiramente o vetor de bits do Filtro de Bloom; a Tabela Hash só é acionada caso o filtro indique que o elemento possivelmente existe. Para embasar as decisões de projeto e observar o comportamento do sistema, foram conduzidos experimentos quantitativos submetendo a aplicação a cargas de 1.000, 10.000 e 100.000 registros gerados aleatoriamente. A avaliação experimental concentrou-se na medição do tempo médio de consulta (comparando o sistema com e sem a atuação do Filtro de Bloom), além de quantificar a taxa de falsos positivos e as consultas evitadas na estrutura principal. Por meio da análise desses resultados, o relatório justifica tecnicamente o dimensionamento do vetor de bits, a quantidade de funções hash utilizadas e o fator de carga adotado para a resolução de colisões, determinando os cenários onde a hibridização algorítmica se mostra mais vantajosa.

Palavras-chave: Tabela Hash; Filtro de Bloom; Estruturas de Dados; Desempenho Computacional; Falsos Positivos.

1. INTRODUÇÃO

O crescimento exponencial do volume de dados gerenciados por sistemas de informação modernos elevou a eficiência das operações de busca a um patamar crítico na Ciência da Computação. Em aplicações de larga escala, responder instantaneamente se um registro existe ou não em um repositório é um fator determinante para a usabilidade de um software. Neste contexto, o presente relatório aborda o desenvolvimento de um Sistema de Verificação de Cadastro de Usuários, cujo propósito hipotético é gerenciar a base de uma empresa com milhões de usuários cadastrados e responder rapidamente à pergunta: *"Este usuário já está cadastrado?"*.

Para atender a essa demanda com precisão, a literatura clássica aponta a Tabela Hash como uma das estruturas fundamentais para o armazenamento e a recuperação exata dos elementos. Segundo Cormen *et al.* (2012), uma tabela de dispersão (*hash table*) é uma estrutura de dados que generaliza a noção de um *array* comum, permitindo que a busca, a inserção e a remoção ocorram, no caso médio, em tempo constante $O(1)$. O funcionamento baseia-se em uma função matemática de espalhamento ($h(k)$) que mapeia o universo de chaves possíveis para os índices de um vetor. Contudo, conforme adverte Knuth (1998), a limitação de espaço do vetor gera inevitáveis colisões, ou seja, ocasiões em que duas chaves distintas resultam no mesmo índice, exigindo a implementação de mecanismos de resolução, tais como o encadeamento externo ou o endereçamento aberto.

Apesar da eficiência assintótica das tabelas hash, o acesso contínuo a blocos de memória para verificar chaves que sequer foram cadastradas gera desperdício de processamento. Para mitigar esse custo, Burton Howard Bloom propôs o Filtro de Bloom (BLOOM, 1970), configurado no escopo deste projeto como uma estrutura probabilística para acelerar consultas de existência. Conforme explicitado por Broder e Mitzenmacher (2004), o filtro opera sobre um vetor de m bits (inicialmente configurados como 0) e utiliza k funções hash independentes. Ao testar a pertinência de uma chave, a estrutura é capaz de afirmar categoricamente quando um elemento *"definitivamente não existe"*, ou apontar que ele *"possivelmente existe"*.

A grande contrapartida teórica dessa barreira inicial é a ocorrência de falsos positivos, cuja taxa percentual é matematicamente controlada pela relação entre o tamanho do vetor de bits e a quantidade de chaves inseridas (BRODER; MITZENMACHER, 2004). Assim, a

proposta operacional deste sistema assenta-se na hibridização: o Filtro de Bloom atua interceptando as buscas na memória e caso a chave passe pelo filtro, deve-se consultar a Tabela Hash para a verificação exata.

Desse modo, o desenvolvimento deste trabalho justifica-se pela necessidade de demonstrar que a engenharia de software de alto desempenho raramente depende de uma única estrutura perfeita, mas sim da integração inteligente de *trade-offs* algorítmicos. Para fins de engenharia, interceptar requisições inválidas logo na camada probabilística poupa acessos custosos à estrutura principal. Portanto, conduzir este estudo medindo o impacto dessa união sob a ótica do consumo de memória, do tempo de execução e da taxa de falsos positivos torna-se essencial. Sendo assim, o trabalho capacita a equipe a tomar decisões de projeto pautadas em análise quantitativa e experimental, abandonando o empirismo vago em prol de justificativas matemáticas sólidas.

2. OBJETIVOS

2.2. Objetivo Geral

- Desenvolver um sistema capaz de armazenar e consultar grandes volumes de dados utilizando duas estruturas complementares: Tabela Hash e Filtro de Bloom.

2.1. Objetivos Específicos

- Implementar manualmente uma função hash e o tratamento de colisões da Tabela Hash;
- Implementar manualmente o vetor de bits do Filtro de Bloom e dimensioná-lo adequadamente;
- Analisar o impacto do uso combinado dessas estruturas em termos de desempenho, consumo de memória e taxa de falsos positivos;
- Executar testes de experimentação com conjuntos de 1.000, 10.000 e 100.000 registros.

3. MATERIAL E MÉTODOS

O desenvolvimento e os experimentos seguiram as especificações técnicas estabelecidas. O sistema foi integralmente desenvolvido na linguagem C. Como restrição metodológica, não foram utilizadas bibliotecas prontas de Tabela Hash ou Bloom Filter.

A arquitetura do sistema seguiu a divisão de responsabilidades da equipe:

- **Tabela Hash:** Foi implementada com suporte a inserção e busca, adotando a técnica de encadeamento externo para o tratamento de colisões. Para garantir um fator de carga ótimo nos experimentos, a tabela foi alocada dinamicamente com o tamanho de 100.009 posições (um número primo), utilizando o algoritmo DJB2 para o espalhamento.
- **Filtro de Bloom:** O dimensionamento não utilizou valores arbitrários. Para garantir uma alocação adequada de memória e uma precisão controlada de 1% ($p = 0.01$) de falsos positivos, o tamanho do vetor de bits (m) e a quantidade de funções hash (k) foram calculados estritamente pelas seguintes equações matemáticas, onde n é a quantidade esperada de elementos:

- Tamanho do vetor: $m = - \frac{n \ln p}{(\ln 2)^2}$

- Número de funções hash: $k = \frac{m}{n} \ln 2$

A estrutura do vetor de bits foi implementada por meio de um vetor de `unsigned char`, no qual cada posição armazena 8 bits manipulados individualmente através de operadores bitwise: o operador OR (`|`) é utilizado na operação de inserção, para ligar um bit sem alterar os demais já armazenados naquele byte, e o operador AND (`&`) é utilizado na operação de consulta, para testar isoladamente o estado de um bit específico. Essa escolha de representação reduz substancialmente o consumo de memória em comparação a uma representação ingênua, na qual cada bit seria armazenado em uma variável do tipo `int` (4 bytes); ao empacotar 8 bits por byte, o consumo de memória é reduzido em até 32 vezes em relação a essa representação ingênua, preservando a eficiência de memória que constitui a principal vantagem teórica do Filtro de Bloom.. Para a geração das k posições de cada elemento, foram combinadas duas funções hash independentes, DJB2 e SDBM, por meio da técnica de Double Hashing, segundo a fórmula `posição_i = (hash1 + i × hash2) mod m`.

O fluxo de consulta obrigatório foi estruturado em quatro etapas:

1. Consultar o Filtro de Bloom;
2. Se o filtro indicar "definitivamente não existe", o sistema retorna imediatamente;
3. Caso indique "possivelmente existe", a consulta é repassada à Tabela Hash;
4. O sistema informa o resultado final ao usuário.

Para a avaliação de desempenho, foram gerados nomes aleatoriamente no formato de 8 caracteres (letras minúsculas) seguidos de 3 números. Os experimentos coletaram estatísticas em cenários controlados, medindo a quantidade de elementos armazenados, tempo médio de consulta, e taxas de falsos positivos.

- **Instruções de Compilação e Execução:**

Os códigos-fonte do projeto estão concentrados no diretório `src/` e os arquivos de teste no diretório `data/`. Para compilar o sistema, é necessário utilizar o compilador GCC executando o comando abaixo a partir da raiz do projeto, garantindo a linkagem da biblioteca matemática (`-lm`):

```
gcc -Wall -Wextra -o sistema_usuarios src/main.c src/hash.c
src/bloom.c -lm
```

Em seguida, o sistema é iniciado executando o binário gerado (`./sistema_usuarios` no Linux/Mac ou `sistema_usuarios.exe` no Windows).

4. RESULTADOS E DISCUSSÃO

4.1. Análise de Colisões na Tabela Hash

A tabela a seguir demonstra o comportamento interno da Tabela Hash durante as inserções da massa de dados.

Métrica Analisada	Cenário 1	Cenário 2	Cenário 3
Tamanho do Vetor (M)	1009	10.009	100.009
Elementos Inseridos (N)	1001*	10.001*	100.001*
Fator de Carga (N/M)	0.99	1.00	1.00

Índices Ocupados	623	6361	63241
Pior Caso (Maior Lista)	4	6	7

Tabela 1: Análise do comportamento da Tabela Hash.

***Nota sobre a volumetria dos experimentos:** Os resultados quantitativos apresentam um acréscimo de uma unidade em relação aos lotes nominais solicitados (1.001, 10.001 e 100.001 elementos). Este comportamento advém da metodologia de testes adotada: após a carga massiva de strings voláteis geradas randomicamente, inseriu-se um registro estático de controle ("*testando123*"). A inclusão deste elemento extra foi fundamental para viabilizar a posterior validação unitária da função *hash_buscar* sob condições extremas de preenchimento, assegurando que o algoritmo de busca linear nas listas encadeadas permanecesse funcional e eficiente mesmo sob alto fator de carga.

Como observado na tabela acima, a estratégia de dimensionar a estrutura para manter o Fator de Carga sempre próximo a 1.0 resultou em uma eficiência excelente. Mesmo no cenário de alto estresse (100.000 elementos), a maior lista encadeada gerada em uma colisão possuía apenas 7 elementos. Isso comprova que a função DJB2 realizou um espalhamento uniforme das chaves e que a busca dentro da Hash está operando muito próxima da complexidade de tempo constante $O(1)$.

4.2. Avaliação de Desempenho e Filtro de Bloom

Para avaliar a influência direta do Filtro de Bloom, cronometrou-se o tempo necessário para consultar milhares de registros.

Antes da análise de desempenho, a Tabela 2 apresenta os valores de dimensionamento do Filtro de Bloom calculados pelas equações matemáticas para cada cenário experimental.

Métrica Analisada	Cenário 1	Cenário 2	Cenário 3
Elementos Esperados (n)	1.000	10.000	100.000
Tamanho do Vetor (m, em bits)	9.586	95.851	958.506
Memória Alocada (bytes)	1.199	11.982	119.814
Funções Hash (k)	7	7	7

Tabela 2: Dimensionamento do Filtro de Bloom nos três cenários experimentais.

Observa-se que o valor de k permaneceu estável (igual a 7) nos três cenários, mesmo com n variando em ordens de grandeza distintas. Isso ocorre porque k depende da razão m/n , e

como m foi dimensionado para crescer proporcionalmente a n , essa razão se manteve praticamente constante, conforme previsto pela formulação teórica.

Métrica Analisada	Cenário 1	Cenário 2	Cenário 3
Quantidade (N)	1.000	10.000	100.000
Tempo sem Bloom (ms)	0.4620	2.7250	21.7960
Tempo com Bloom (ms)	0.3730	1.9710	19.2110
Falsos Positivos	40	367	2566
Taxa de Falsos Positivos (%)	4,000	3,670	2,566

Tabela 3: Análise da influência do Filtro de Bloom.

Com base nos dados experimentais coletados, é possível responder aos questionamentos centrais do projeto:

1. O Filtro de Bloom reduziu consultas na Tabela Hash?

Sim. O Filtro de Bloom atuou como um excelente escudo protetor da memória principal. A estrutura barrou imediatamente a grande maioria das buscas por usuários inexistentes, poupando a Tabela Hash de calcular índices e varrer listas encadeadas, repassando à estrutura secundária apenas a pequena margem de falsos positivos descrita na tabela.

2. Como o tamanho do vetor impactou os resultados?

O dimensionamento dinâmico e matemático do vetor (aumentando proporcionalmente ao número de elementos) permitiu que a taxa de falsos positivos se mantivesse estável. O uso das fórmulas matemáticas evitou que o vetor se saturasse com muitos bits ligados, fato que ocorreria inevitavelmente se um vetor pequeno de tamanho fixo fosse usado para armazenar 100.000 registros.

3. O número de funções hash alterou a precisão?

Sim, o cálculo exato do número de funções hash (k) foi vital. A utilização da equação $k = (m/n) \cdot \ln 2$ determinou a quantidade ideal de bits a serem ligados por elemento. Utilizar funções de menos geraria colisões de chaves no filtro, enquanto usar funções demais

preencheria rapidamente o vetor de bits, o que em ambos os casos elevaria os falsos positivos e destruiria a precisão do filtro.

4. Em qual cenário o Bloom foi mais vantajoso?

O uso do Filtro de Bloom demonstrou-se amplamente mais vantajoso no cenário de 100.000 registros. Em baixos volumes, o processamento de k funções hash extras pode rivalizar com o tempo de uma busca direta e rápida na Hash. Contudo, em cenários de alta carga, o alto custo temporal de acessar repetidamente a memória para procurar elementos que não existem (*cache miss*) faz com que a barreira preliminar do Bloom seja indispensável para manter a performance do software.

5. CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto validou experimentalmente os conceitos teóricos de estruturas de dados probabilísticas atuando em conjunto com estruturas de busca exatas. Ao longo do estudo, comprovou-se que a Tabela Hash é robusta e extremamente eficiente na recuperação de dados quando dimensionada corretamente, mantendo as colisões e o fator de carga sob controle num cenário de alto volume.

No entanto, o maior diferencial da implementação residiu na adição do Filtro de Bloom. Ao aplicar o dimensionamento matemático rigoroso das equações propostas na literatura, a estrutura cumpriu perfeitamente o seu papel de atuar como uma barreira inicial, garantindo uma taxa de erro tolerável e estática sem causar desperdício excessivo de memória RAM.

Conclui-se, portanto, que a hibridização algorítmica é altamente benéfica. O uso do Filtro de Bloom reduziu o desperdício de processamento para pesquisas de chaves inexistentes, consolidando um Sistema de Consulta capaz de lidar com centenas de milhares de usuários com tempos de resposta da ordem de milissegundos.

6. REFERÊNCIAS

BLOOM, Burton H. Space/time trade-offs in hash coding with allowable errors.

Communications of the ACM, v. 13, n. 7, p. 422-426, 1970.

BRODER, Andrei; MITZENMACHER, Michael. Network applications of bloom filters: A survey. *Internet Mathematics*, v. 1, n. 4, p. 485-509, 2004.

CORMEN, Thomas H. et al. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

KNUTH, Donald Ervin. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Reading: Addison-Wesley, 1998.